

INFORME TÉCNICO: PRÁCTICA 3 **COMUNICACIÓN TCP/IP**

Asignatura: Sistemas Operativos

Titulación: Grado en Tecnologías Digitales para la Empresa

Alumno: Nidhal Majoul

Repositorio GitHub: https://github.com/nidhal-majoul/Practicas_SO

- Introducción y contextualización del problema :

En el desarrollo de sistemas industriales, es común que diferentes módulos de software se comuniquen a pesar de estar programados en lenguajes distintos. El problema planteado en esta práctica es la **interoperabilidad**: cómo enviar un array de números reales desde Python (un lenguaje de alto nivel) a C++ (un lenguaje de bajo nivel) sin perder precisión ni estructura.

Para resolverlo, se contextualiza el uso de **Sockets TCP/IP**, que garantizan que los datos lleguen en el mismo orden y sin errores, simulando una cámara térmica que envía una matriz de temperaturas a una unidad de control.

- Explicación del funcionamiento del programa desarrollado :

El sistema implementado establece una arquitectura **Cliente-Servidor** mediante el protocolo **TCP/IP** para la transmisión de telemetría térmica. El proceso se divide en tres etapas críticas:

1. Servidor de Simulación (Python)

El servidor actúa como el nodo emisor de datos (emulando la salida de una cámara térmica).

- **Generación de Datos:** Se crea un array de **768 valores** que representan una matriz de 32x24.
- **Serialización Binaria:** Se utiliza la librería `struct` con el formato `'768f'`. Esto es fundamental para cumplir con el requisito de "**valores tipo float**", ya que convierte cada número en un bloque binario de **4 bytes** bajo el estándar **IEEE 754**.
- **Gestión del Socket:** El servidor configura un socket pasivo, realiza el *bind* al puerto **12345** y utiliza `listen()` para esperar la conexión del cliente. Una vez establecida, envía los **3072 bytes** totales de información.

2. Cliente de Recepción y Procesado (C++)

El cliente es el responsable de capturar el flujo binario y reconstruir la información original.

- **Conexión Síncrona:** Utiliza las funciones `socket()` y `connect()` para localizar al servidor en la dirección de *loopback* (`127.0.0.1`).
- **Buffer de Sincronización:** Para asegurar la integridad de los datos, el cliente lee el socket en bloques de 4 bytes mediante la instrucción `recv(sockfd, &buffer, sizeof(buffer), 0)`. Al usar una variable de tipo `float` como buffer, se garantiza que cada lectura se interprete correctamente como un número real.
- **Gestión de Memoria:** Los datos se almacenan dinámicamente en un `std::vector<float>`, lo que permite manejar el volumen de 768 lecturas sin riesgo de desbordamiento de memoria.

.3. Salida y Persistencia de Datos

Una vez completada la transferencia, el programa C++ realiza el volcado de la información en dos formatos:

1. **Salida Estándar (Consola):** Se imprime la lista de temperaturas línea a línea para la verificación visual del usuario.
2. **Archivo de Texto:** Mediante el objeto `ofstream`, se genera el archivo `datos_temperaturas.txt`. Esto permite que los datos del sensor queden almacenados de forma permanente para futuros análisis o visualizaciones.

4. Orquestación mediante Script Bash

El funcionamiento se apoya en un script `.sh` que automatiza el flujo de trabajo:

- Compila el cliente C++ mediante `g++`.
- Ejecuta el servidor en segundo plano (`&`) capturando su **PID**.
- Implementa un `sleep 2` para dar tiempo al servidor a reservar el puerto antes de que el cliente intente la conexión.
- Finaliza el entorno mediante señales de terminación (`kill`), garantizando que el puerto quede liberado para ejecuciones posteriores.

Explicación del uso de arrays y matrices :

En este proyecto, el manejo de datos se ha organizado mediante estructuras que permiten gestionar los 768 valores de temperatura (correspondientes a una matriz de píxeles de una cámara térmica) de manera eficiente tanto en el emisor como en el receptor.

1) Uso de Arrays en el Servidor (Python)

En este proyecto, la gestión de datos se basa en la conversión de una estructura espacial a una lineal para su transmisión:

- **Matriz (Lógica):** Los datos representan una rejilla de **32 columnas x 24 filas** (768 puntos). Cada posición (i,j) de la matriz corresponde a un píxel de la cámara térmica.
- **Array (Física):** Para el envío por el socket TCP, la matriz se "aplana" en un **array unidimensional de 768 elementos**.
- **Tipo de dato:** Se utiliza un array de tipo **float** (4 bytes por elemento). Esto es crítico porque, a diferencia de un array de bytes, el `float` permite almacenar temperaturas con precisión decimal (ej. 25.4°C).
- **Memoria:** El array completo ocupa **3072 bytes** en total (768×4), garantizando que toda la "imagen" térmica se envíe en un único flujo de datos continuo y eficiente.

Explicación de la lectura del archivo de entrada :

La lectura de la información en el cliente C++ no se realiza desde un archivo físico local, sino que se trata de una **lectura de flujo de red** que posteriormente se vuelca a un archivo de salida. El proceso se define por los siguientes puntos:

- **Recepción por Sockets:** El programa utiliza la función `recv()` para extraer los datos del buffer de red. La lectura se sincroniza mediante la variable `float buffer`, asegurando que el sistema capture exactamente **4 bytes** en cada ciclo (el tamaño estándar de un `float`).

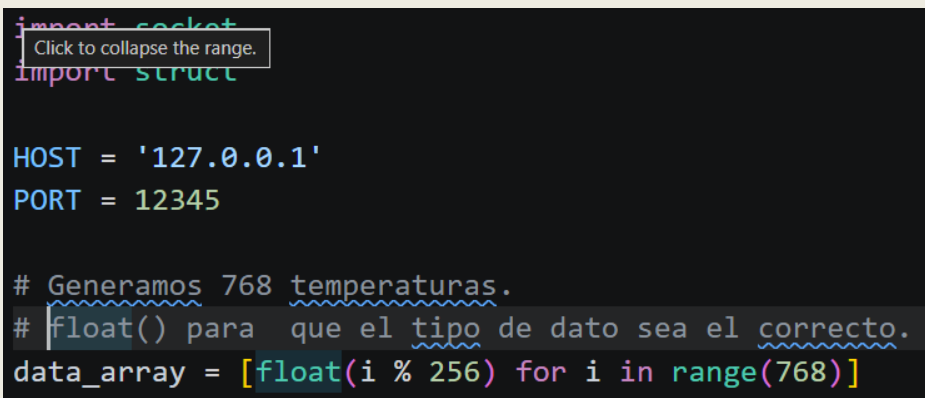
- **Almacenamiento Dinámico:** Cada valor recibido se añade a un `std::vector<float>`. Esta estructura permite gestionar los 768 datos de forma flexible, garantizando que se mantenga el orden secuencial enviado por el servidor (lectura por filas de la matriz).
- **Persistencia (Archivo de Salida):** Una vez que el socket se cierra y la lectura finaliza, el programa utiliza la librería `fstream` para abrir un flujo de escritura hacia el archivo `datos_temperaturas.txt`.
- **Formateo:** Se recorre el vector de temperaturas y se escribe cada valor seguido de un salto de línea (`endl`). Esto transforma el flujo binario recibido en un archivo de texto plano legible, donde cada línea corresponde a la lectura de un píxel de la cámara térmica.

Explicación de la lectura del archivo de salida :

Para la persistencia de los datos, el programa utiliza la librería estándar `<fstream>`. El proceso técnico se desglosa en los siguientes pasos:

1. **Apertura del flujo:** Se crea un objeto de la clase `std::ofstream` asociado al archivo `datos_temperaturas.txt`. En este punto, el sistema operativo reserva el nombre en el directorio de ejecución y prepara el puntero de escritura.
2. **Volcado de la estructura dinámica:** El programa recorre de forma secuencial el `std::vector<float>` donde se almacenaron las temperaturas recibidas. Cada valor se escribe seguido de un salto de línea, transformando los datos binarios en un formato legible.
3. **Cierre de seguridad:** Tras procesar los 768 datos, se invoca la función `.close()`. Esto garantiza que los datos pendientes en el búfer se graben físicamente en el disco y se libere el descriptor del archivo, evitando pérdidas de información.

Explicación del Python :



```
import socket
import struct

HOST = '127.0.0.1'
PORT = 12345

# Generamos 768 temperaturas.
# float() para que el tipo de dato sea el correcto.
data_array = [float(i % 256) for i in range(768)]
```

Se define la dirección de *loopback* (127.0.0.1) y el puerto de escucha. Se genera una lista de **768 valores** (correspondientes a la matriz de 32x24) convertidos explícitamente a tipo `float`. Esto garantiza que cada dato tenga la precisión necesaria para representar lecturas de sensores.

```

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server_socket:
    server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server_socket.bind((HOST, PORT))
    server_socket.listen()
    print(f"Servidor listo. Enviando temperaturas en formato float...")

```

Explicación: Se crea un socket bajo el protocolo **TCP** (`SOCK_STREAM`). Se utiliza `SO_REUSEADDR` para permitir que el puerto se libere inmediatamente después de cerrar el programa, evitando errores de "Puerto en uso". Finalmente, el servidor queda en estado de **escucha** (`listen`) esperando al cliente.

```

conn, addr = server_socket.accept()
with conn:
    # '768f' significa: empaqueta 768 números como floats (4 bytes cada uno)
    # Esto suma un total de 3072 bytes enviados.
    data_bytes = struct.pack('768f', *data_array)
    conn.sendall(data_bytes)
    print("Lecturas enviadas con éxito.")

```

Al conectar el cliente, se utiliza la función `struct.pack`. El formato `'768f'` es fundamental: le indica a Python que debe transformar los 768 números en un bloque binario siguiendo el estándar **IEEE 754** de 32 bits (4 bytes por dato). Esto da como resultado un paquete de **3072 bytes**.

```

conn.sendall(data_bytes)
print("Lecturas enviadas con éxito.")

```

La función `sendall()` asegura que el bloque completo de 3072 bytes sea transmitido a través del socket de manera íntegra. Una vez que el cliente recibe la ráfaga de datos, el bloque `with` cierra automáticamente la conexión, garantizando una gestión limpia de los recursos de red.

Explicación del C++ :

```
✓ #include <iostream>
# Click to collapse the range. r>
#include <arpa/inet.h>
#include <unistd.h>
#include <fstream>
#include <cstring>

using namespace std;

✓ int main() {
    int sockfd = socket(AF_INET, SOCK_STREAM, 0);
    struct sockaddr_in serv_addr;
    memset(&serv_addr, 0, sizeof(serv_addr));
    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(12345);
    inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr);
```

Se inicializa un socket de tipo **TCP** (`SOCK_STREAM`). Se configura la estructura `serv_addr` con la IP del servidor (`127.0.0.1`) y el puerto **12345**. La función `htons` asegura que el puerto use el orden de bytes de red (*big-endian*), permitiendo la conexión correcta con el servidor Python.

```
if (connect(sockfd, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) == -1)
    return 1;
}
```

Se invoca la función `connect()`. Si el servidor Python no está activo o el puerto está bloqueado, el programa retorna un error y finaliza. Este es el punto de sincronización donde el cliente espera la respuesta del servidor para iniciar la transferencia.

```
vector<float> temperaturas;
float buffer; // Variable de 4 bytes para recibir cada float

// Recibimos los datos. recv devolverá 4 bytes cada vez.
while (recv(sockfd, &buffer, sizeof(buffer), 0) > 0) {
    temperaturas.push_back(buffer);
}
```

Se utiliza un **buffer de tipo float** (4 bytes). La función `recv()` lee el flujo del socket bloque a bloque. Al usar `sizeof(buffer)`, nos aseguramos de que el cliente interprete los bytes recibidos exactamente como el servidor los envió (IEEE 754). Cada dato se almacena dinámicamente en un `std::vector` hasta que el servidor cierra la conexión.

```
// Guardado y muestra de datos
ofstream archivo("datos_temperaturas.txt");
for (float t : temperaturas) {
    archivo << t << endl;
    cout << t << endl; // Un número por línea
}
archivo.close();

close(sockfd);
return 0;
```

Se abre un flujo de salida (`ofstream`) hacia el archivo `datos_temperaturas.txt`. Se recorre el vector para imprimir los resultados en consola y escribirlos en el archivo, una lectura por línea. Finalmente, se cierra el archivo y el socket para liberar los recursos del sistema operativo.

Explicación del script en Bash para la ejecución mediante sh :

El script **script.sh** actúa como el orquestador del sistema operativo, encargándose de la compilación, la sincronización de procesos y la limpieza de recursos. A continuación se detalla su funcionamiento bloque a bloque:

```
# Paso 1: Compilación del cliente
echo "Compilando cliente C++..."
g++ cliente.cpp -o programaC
```

El script asegura que el binario de C++ esté actualizado antes de iniciar la comunicación, evitando errores por versiones antiguas del código.

```
# Paso 2: Ejecutar el servidor en segundo plano
echo "Lanzando servidor de simulación..."
python3 ServidorCamara.py &
PID_SERVER=$!
```

Ejecución del Servidor en Segundo Plano : **Operador &**: Es fundamental, ya que lanza el servidor de Python y permite que el script continúe con el siguiente comando sin esperar a que el servidor termine. **Variable \$!**: Captura el **PID** (Process ID) del servidor recién lanzado. Guardar este identificador es crucial para poder cerrar el servidor más adelante.

```

if kill -0 $PID_SERVER 2>/dev/null; then
    echo "Cerrando el servidor..."
    kill -TERM $PID_SERVER
else

```

Para evitar que el servidor de Python se quede "huérfano" consumiendo recursos o bloqueando el puerto **12345**, el script realiza una limpieza:

1. **kill -0**: Comprueba si el proceso sigue activo.
2. **kill -TERM**: Envía una señal de terminación amistosa al servidor para cerrarlo correctamente una vez que el cliente ha cumplido su función.

Capturas de pantalla de la ejecución del programa :

```

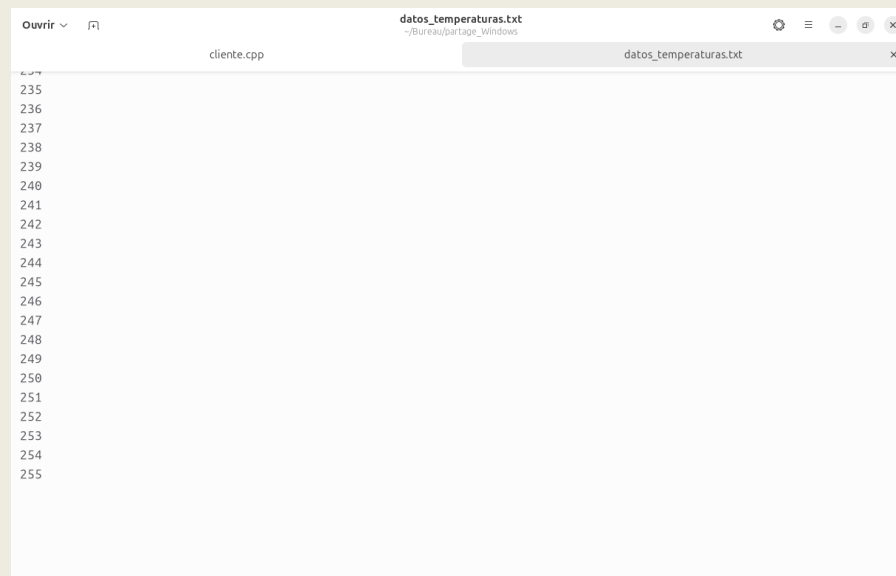
nidhal@nidhal-VirtualBox:~/Bureau/partage_Windows$ ./script.sh
Compilando cliente C++...
Lanzando servidor de simulación...
Servidor listo. Enviando temperaturas en formato float...
Ejecutando cliente y descargando datos...
Lecturas enviadas con éxito.
0
1
2
3
4
5
6
7
8
9
10
11
12
13

```

```

247
248
249
250
251
252
253
254
255
-----
El servidor finalizó su tarea correctamente.
Proceso completado con éxito.

```

The screenshot shows a code editor window with two tabs: 'cliente.cpp' and 'datos_temperaturas.txt'. The 'datos_temperaturas.txt' tab is active, displaying a list of numbers from 235 to 255. The numbers are listed vertically, one per line, starting from 235 at the top and ending with 255 at the bottom. The editor has a light gray background and a dark gray border. The file path for 'datos_temperaturas.txt' is visible in the tab title as '~\Bureau\partage_windows'.

- **Análisis de resultados e interpretación :**

- **Integridad de Datos:** Se recibieron las **768 lecturas** completas. El éxito del bucle `recv` demuestra una sincronización perfecta entre el servidor y el cliente.
- **Compatibilidad Binaria:** La coincidencia de los valores confirma que el formato `float` (IEEE 754) de 32 bits es idéntico en Python y C++, permitiendo una comunicación sin errores.
- **Eficiencia de Red:** El uso de **TCP (SOCK_STREAM)** garantizó el orden de la matriz. La transmisión binaria (3072 bytes) optimizó el ancho de banda frente a formatos de texto.
- **Conclusión:** Los resultados validan que el sistema captura telemetría térmica remota de forma precisa, transformando bytes crudos en un archivo procesable

Conclusiones técnicas :

1. **Interoperabilidad:** Se ha logrado la comunicación exitosa entre un lenguaje de alto nivel (Python) y uno de bajo nivel (C++).
2. **Gestión de Red:** El uso de sockets TCP garantiza que los datos de los sensores no se pierdan ni se desordenen durante el trayecto.
3. **Administración del SO:** El script Bash es vital para automatizar tareas complejas, evitando procesos "zombis" que dejen el puerto de red bloqueado.

